

Software Requirement Specification (SRS)

“Arooohi” ***An exclusive, student-to-student ride-sharing network***

Software Requirements Specification

Prepared by

Student ID	Name
23201330	Ahnaf Bin Zakaria
24101167	Mujtahidul Hasan Sami
22101569	Aminul Islam
22101476	Sayed Hasanul Zayed

1. Introduction

1.1 Purpose

This Software Requirement Specification (SRS) document outlines the requirements for developing “Arooohi” - an exclusive student-to-student ride-sharing network. The primary goal of this application is to provide secure, low-cost commutes, generate passive income for drivers, and offer specialized matching like female-to-female rides for enhanced campus safety, utilizing FastAPI for the backend.

1.2 Scope

The scope of this project includes the design, development, and deployment of a ride-sharing application catering to:

- Student Passengers (Riders) who can book rides, split costs, share trusted contact tracking, and utilize an SOS button.
- Student Drivers who can manage their earnings dashboard, accept rides, and view peak-hour surges.
- Administrators/Campus Moderators who oversee system operations, handle complaints, and verify students.

1.3 Definitions, Acronyms, and Abbreviations

- **FastAPI:** A modern, fast web framework for building APIs with Python.
- **GPS:** Global Positioning System.
- **SOS:** Emergency distress signal.
- **NID:** National Identity Card.

1.4 References

- FastAPI Documentation
- React/React Native Documentation
- bKash API Documentation

1.5 Overview

Section 2 provides an overall description of the product, including user classes and features. Section 3 details the system requirements (functional, non-functional, external interfaces). Section 4 outlines the tools and technologies. Section 5 contains the class diagram logic. Section 6 covers compatibility. Section 7 presents the development plan. Section 8 displays the implementation features. Sections 9, 10, and 11 cover challenges, conclusion, and references.

2. Overall Description

2.1 Product Perspective

Arooohi is a standalone mobile and web application designed exclusively for the student ecosystem. It integrates with external APIs such as bKash for payments and mapping services for live GPS tracking.

2.2 Product Features

1. BRACU Student Verification — only verified @g.bracu.ac.bd emails can register
2. Live GPS Ride Tracking — real-time location sharing with trusted contacts
3. Female-Only Ride Mode — women can filter to match only with female riders/drivers
4. In-App SOS Button — instant alert to campus security + emergency contacts
5. Ride Cost Splitter — auto-calculates and splits fare among passengers
6. Campus Zone Smart Matching — matches riders heading to same area/gate/building
7. Driver Rating & Review System — post-ride ratings for both drivers and passengers
8. Scheduled Ride Booking — book rides in advance (e.g., for 8 AM class)
9. Wallet & bKash Integration — cashless payment inside the app
10. Ride History & Receipt Log — downloadable trip records for personal tracking
11. Driver Vehicle Verification — upload NID, bike/car registration, and license
12. Trusted Contact Sharing — auto-share ride details to a saved guardian/friend
13. Peak Hour Surge Indicator — shows busy times to help plan rides better
14. Campus Pickup Hotspots — predefined pickup points (Gate 1, Cafe, Library, etc.)
15. Ride Chat (In-App Messaging) — communicate without sharing personal numbers
16. Driver Earnings Dashboard — students track how much they've earned each week
17. Admin Complaint Panel — report misconduct; reviewed by a campus moderator
18. Ride Cancellation Policy & Penalty — reduces last-minute no-shows
19. Multi-Stop Ride Support — one driver drops multiple passengers at different stops
20. Eco/Footprint Tracker — shows CO2 saved by carpooling vs. solo rides

2.3 User Classes and Characteristics

- Student Riders: Students seeking safe, affordable rides. Require an intuitive interface and strong safety features.
- Student Drivers: Verified students offering rides for passive income. Need features to track earnings, plan routes, and manage multiple stops.
- Administrators: Campus moderators responsible for handling SOS alerts and reviewing misconduct complaints.

2.4 Operating Environment

- Web Application: Accessible on modern web browsers (Chrome, Firefox, Edge, Safari).
- Mobile Compatibility: Responsive design for smartphones and tablets.
- Server-Side: FastAPI running in a containerized or cloud environment.
- Database: PostgreSQL (with PostGIS) or MongoDB cluster.

2.5 Constraints

- Regulatory Compliance: Must adhere to university privacy and data policies.
- Security: Must implement encryption at rest and in transit.
- Performance: The system must support real-time GPS tracking with minimal latency.

2.6 Assumptions and Dependencies

- Users have a stable internet connection and active GPS.
- Third-party services for payment (bKash) and mapping are available and reliable.

3. System Requirements

3.1 Functional Requirements

3.1.1 Authentication & Security

FR-1: The system shall enforce @g.bracu.ac.bd email verification for registration.

FR-2: The system shall allow upload and manual/automated verification of NID, driving license, and vehicle registration for drivers.

FR-3: The system shall execute an SOS alert to campus security and trusted contacts when triggered.

3.1.2 Ride Management & Matching

FR-4: The system shall allow female users to toggle a mode restricting matches to other female users.

FR-5: The system shall group riders heading to predefined hotspots via Campus Zone Smart Matching.

FR-6: The system shall support scheduled bookings and multi-stop capabilities for a single ride.

3.1.3 Logistics & Tracking

FR-7: The system shall track and share real-time GPS coordinates with trusted contacts.

FR-8: The system shall provide in-app messaging to facilitate driver-rider communication.

FR-9: The system shall dynamically calculate and display peak hour surges.

3.1.4 Payments & Dashboard

FR-10: The system shall integrate bKash and an in-app wallet to split and process fares.

FR-11: The system shall maintain a driver earnings dashboard and downloadable trip logs.

FR-12: The system shall track the CO2 emissions saved and display it on the Eco Tracker.

3.2 Non-Functional Requirements

NFR-1 (Performance): Live GPS and SOS alerts must be processed within strict latency limits (e.g., < 2 seconds).

NFR-2 (Security): Sensitive data like NIDs and passwords must be encrypted using industry standards.

NFR-3 (Reliability): The system must maintain 99.9% uptime to ensure commuter safety at all hours.

NFR-4 (Maintainability): The FastAPI backend must be fully documented using auto-generated Swagger UI.

NFR-5 (Scalability): Architecture must support horizontal scaling during morning/evening campus rush hours.

3.3 External Interface Requirements

3.3.1 User Interfaces: Responsive web interface and native mobile viewports with separate driver/rider dashboards.

3.3.2 Hardware Interfaces: GPS hardware on user smartphones for location tracking.

3.3.3 Software Interfaces: bKash payment gateway API; Mapbox/Google Maps API for routing.

3.3.4 Communication Interfaces: RESTful APIs and WebSockets for real-time frontend-backend communication.

4. Tools and Technologies

4.1 Technology Stack Components

- Frontend: React.js (Web) and React Native (Mobile, for future purpose) for cross-platform, responsive interfaces.

- Backend: FastAPI (Python) for asynchronous, high-speed API endpoints and WebSocket handling.

- Database: PostgreSQL (with PostGIS extension) for reliable handling of geospatial data and transactions, or MongoDB.

- Storage: AWS S3 or similar for storing verification documents securely.

4.2 High-Level Architecture

- Presentation Layer: Handles user location updates, form submissions, and UI state management.
- Business Logic Layer: FastAPI application routing, ride-matching algorithms, and surge pricing logic.
- Data Layer: Persistent storage of user data, ride logs, and financial transactions.

5. Class Diagram

The system structure comprises the following core entities:

- User (Base Class): attributes (id, name, bracu_email, phone, is_verified); methods (login(), logout(), resetPassword()).
- Rider (Inherits User): attributes (trusted_contacts, wallet_balance); methods (requestRide(), triggerSOS(), splitFare()).
- Driver (Inherits User): attributes (nid, license_no, vehicle_reg, rating, earnings); methods (acceptRide(), startTrip(), endTrip()).
- Ride: attributes (ride_id, source, destination, status, base_fare, surge_multiplier); methods (calculateFare(), assignDriver()).
- Payment: attributes (txn_id, amount, method, status); methods (processbKash(), refund()).
- Admin: methods (verifyDriverDocs(), reviewComplaint(), viewSystemMetrics()).

(Note: As requested by the template instructions, use software like draw.io or Lucidchart to render this into a UML diagram and paste the image here.)

6. Compatibility with System Environment or OS

The Arooohi backend built on FastAPI is OS-agnostic and containerized, ensuring compatibility with any Linux/Windows/macOS server environment (e.g., AWS, Azure, Heroku). The frontend web application is fully compatible with standard modern browsers. The mobile application, built via React Native, is cross-compatible with Android 8.0+ and iOS 12.0+ environments.

7. Tentative Development Plan & Acceptance Criteria

Sprint 1: Core Setup & Authentication (Weeks 1-2)

Features: BRACU Student Verification, Driver Vehicle Verification.
Setup base databases and user schemas.

Sprint 2: Ride Matching & Logistics (Weeks 3-4)

Features: Campus Zone Smart Matching, Campus Pickup Hotspots, Scheduled Ride Booking, Multi-Stop Ride Support.

Sprint 3: Tracking & Safety (Weeks 5-6)

Features: Live GPS Tracking, Female-Only Mode, In-App SOS Button, Trusted Contact Sharing, Ride Chat.

Sprint 4: Payments & Earnings (Weeks 7-8)

Features: Ride Cost Splitter, Wallet & bKash Integration, Driver Earnings Dashboard, Ride History, Peak Hour Surge.

Sprint 5: Quality & Admin Controls (Weeks 9-10)

Features: Driver Rating & Reviews, Admin Complaint Panel, Cancellation Policy & Penalty, Eco/Footprint Tracker.

Acceptance Criteria

- System successfully rejects non-BRACU email registrations.
- Live tracking updates coordinates within 3 seconds.
- SOS button successfully dispatches mock alerts to selected contacts.
- bKash test environment processes split payments accurately.
- Female-only mode restricts visibility completely from male profiles.

8. Implementation: Screenshots per feature

1. BRACU Student Verification

Users enter their @g.bracu.ac.bd email and verify via OTP to access the app.

[Insert Screenshot: 1. BRACU Student Verification]

2. Live GPS Ride Tracking

Map interface displaying the real-time movement of the driver towards the pickup point.

[Insert Screenshot: 2. Live GPS Ride Tracking]

3. Female-Only Ride Mode

A toggle switch on the rider dashboard that filters active drivers to strictly female students.

[Insert Screenshot: 3. Female-Only Ride Mode]

4. In-App SOS Button

A prominent red SOS button that immediately triggers alerts to contacts and campus security.

[Insert Screenshot: 4. In-App SOS Button]

5. Ride Cost Splitter

A payment screen that automatically divides the total ride fare evenly among selected passengers.

[Insert Screenshot: 5. Ride Cost Splitter]

6. Campus Zone Smart Matching

Algorithm results showing riders grouped by their destination zones like 'Gate 1' or 'Library'.

[Insert Screenshot: 6. Campus Zone Smart Matching]

7. Driver Rating & Review System

Post-ride prompt allowing the rider to rate the driver out of 5 stars and leave written feedback.

[Insert Screenshot: 7. Driver Rating & Review System]

8. Scheduled Ride Booking

Calendar and time-picker interface allowing students to pre-book a ride for upcoming classes.

[Insert Screenshot: 8. Scheduled Ride Booking]

9. Wallet & bKash Integration

Digital wallet page showing current balance and a portal to top-up via bKash.

[Insert Screenshot: 9. Wallet & bKash Integration]

10. Ride History & Receipt Log

A detailed list of past trips with options to download PDF receipts for personal records.

[Insert Screenshot: 10. Ride History & Receipt Log]

11. Driver Vehicle Verification

Document upload page for aspiring drivers to submit NID, driving license, and vehicle registration.

[Insert Screenshot: 11. Driver Vehicle Verification]

12. Trusted Contact Sharing

Settings menu where users add emergency contacts to whom live tracking links are automatically sent.

[Insert Screenshot: 12. Trusted Contact Sharing]

13. Peak Hour Surge Indicator

Visual heatmap or multiplier icon on the booking screen warning of increased fares due to high demand.

[Insert Screenshot: 13. Peak Hour Surge Indicator]

14. Campus Pickup Hotspots

Map pins highlighting designated university pickup and drop-off points for easier coordination.

[Insert Screenshot: 14. Campus Pickup Hotspots]

15. Ride Chat (In-App Messaging)

Secure, encrypted chat interface connecting the rider and driver without exposing phone numbers.

[Insert Screenshot: 15. Ride Chat (In-App Messaging)]

16. Driver Earnings Dashboard

Analytical dashboard showing daily/weekly earnings, completed rides, and upcoming payouts.

[Insert Screenshot: 16. Driver Earnings Dashboard]

17. Admin Complaint Panel

Backend interface for moderators to review reported users, read chat logs, and issue bans/warnings.

[Insert Screenshot: 17. Admin Complaint Panel]

18. Ride Cancellation Policy & Penalty

Warning prompt displaying the penalty fee if a user cancels a ride after a driver has been dispatched.

[Insert Screenshot: 18. Ride Cancellation Policy & Penalty]

19. Multi-Stop Ride Support

Route mapping interface showing an optimized path to drop off up to 3 passengers at varying stops.

[Insert Screenshot: 19. Multi-Stop Ride Support]

20. Eco/Footprint Tracker

Gamified statistics page showing grams of CO2 saved by choosing to carpool rather than driving solo.

[Insert Screenshot: 20. Eco/Footprint Tracker]

9. Challenges

- Real-time Synchronization: Maintaining seamless WebSockets for live GPS and chat in fluctuating mobile network areas.
- Complex Fare Algorithms: Calculating dynamic surge pricing and multi-stop fare splits

fairly and instantly.

- Strict Verification: Ensuring the vehicle document validation process is accurate and prevents fraudulent driver accounts.

10. Conclusion

This SRS provides a fully comprehensive outline for the design and development of “Arooohi”. By leveraging FastAPI for a high-performance backend, React Native for cross-platform access, and stringent security protocols, the system ensures a highly secure, efficient, and student-focused ride-sharing ecosystem. Following the outlined Agile sprints and strict acceptance criteria, the project aims to deliver a robust solution that solves campus commuting and safety challenges effectively.

11. References

1. FastAPI Documentation (<https://fastapi.tiangolo.com/>)
2. React / React Native Documentation
3. PostgreSQL and PostGIS Documentation
4. bKash Developer API Guide